Private Cloud Storage System
Using Raspberry Pi

A Report
Submitted to the Council of
the College of Science at the University of
Sulaimani in Partial Fulfillment of the Requirements
for the Bachelor of Science
in Computer Science

by

(Hasty Bast)

(Kozhyar Salah)

(Lawa Dara)

(Dilan Luqamn)

(Dr. Najmadin Wahid)

january, 2026 (English) Xakelêwe, 2726 (Kurdish)

Dedication

We dedicate this project to our parents for their endless support, encouragement, and sacrifices throughout our academic journey. We also extend our sincere gratitude to our instructors and supervisors for their valuable guidance, advice, and continuous support, which made the successful completion of this project possible..

Acknowledgements

We would like to express our sincere gratitude to our project supervisor for the valuable guidance, continuous support, and constructive feedback provided throughout the development of this project. We also thank our instructors and department staff for their assistance and cooperation.

Furthermore, we extend our appreciation to our families and friends for their encouragement, patience, and moral support during the completion of this work.

.

Team members full name

January 29, 2026

Abstract

Cloud storage has become essential for data management, but concerns about security, privacy, and recurring subscription costs remain significant barriers for individuals and small organizations. This project proposes a cost-effective private cloud storage solution using Raspberry Pi as a self-hosted alternative to commercial cloud services. The system is built using Node.js for the backend server, React for the web-based user interface, and implements secure file management with encryption and user authentication. By utilizing Raspberry Pi connected to external storage (such as a hard disk drive), users can expand their storage capacity without recurring fees while maintaining complete control over their data within their local network. The system includes features such as file upload/download, folder management, search functionality, and hash-based integrity verification. Performance testing evaluates upload and download speeds across various file sizes (50KB, 500KB, 1MB, 5MB, and 25MB) with different compression levels. This solution addresses the dual challenges of affordability and data privacy, offering a practical alternative to third-party cloud services for users with limited budgets and high security requirements.

Table of Contents

Dedication ii
Acknowledgements iii
Abstract iv
List of Abbreviations vii
Chapter 1 9
Introduction 9
1.1. Introduction 9
1.2. Problem Statement 10
1.3. Background and Literature review 11
1.4. The Aims of the Project 12
1.5. Report Layout ..... **Error! Bookmark not defined.**
Chapter Two: ..... **Error! Bookmark not defined.**
Chapter Three: ..... **Error! Bookmark not defined.**
Chapter 2 17
Design and Implementation 17
2.1 Introduction 17
Chapter 3 23
Conclusions and Future Directions ..... **Error! Bookmark not defined.**
3.1 Introduction 23
3.2 Conclusions 39
3.3 Future Directions 39
پوخته 40
Bibliography ..... **Error! Bookmark not defined.**

Table of Figures

Figure 1: UoS Logo 9

List of Abbreviations

A/D	**A/D Analog-To-Digital**	**DNI**	**DNI Direct Neural Interface**

Chapter One
Introduction

Chapter 1

Introduction

1.1. Introduction

The exponential growth of digital content has created an unprecedented need for reliable data storage solutions. Families accumulate thousands of photos, videos, documents, and music files over time, while small businesses generate reports, customer data, and project files that require secure, accessible storage. Traditional local storage solutions like external hard drives are vulnerable to physical damage, theft, or hardware failure, potentially resulting in permanent data loss.[1]

Cloud computing has revolutionized data storage by offering remote server-based solutions that provide accessibility from anywhere with an internet connection. Users can access their files across multiple devices, share content with others, and protect against localized disasters. Major cloud providers like Google Drive, Dropbox, and Microsoft OneDrive offer convenient services with automatic synchronization and backup features. However, these services present several challenges:

Figure 1: UoS Logo

1.2. Problem Statement

Cost concerns: Most cloud providers charge monthly or annual subscription fees that increase significantly with storage capacity. For example, storing 2TB of data can cost $10-20 per month, accumulating to hundreds of dollars annually. For families or small businesses with extensive media libraries, these costs become prohibitive.[2]

Privacy and security concerns: Storing sensitive data on third-party servers raises questions about data ownership, privacy policies, and potential unauthorized access. Recent data breaches and concerns about government surveillance have made users increasingly cautious about entrusting personal information to external providers.

Dependency on internet connectivity: Accessing cloud-stored files requires stable internet connections, which may not always be available or reliable, particularly in remote areas or during network outages.

Limited control: Users have minimal control over how their data is stored, managed, or protected on third-party servers, and are subject to terms of service that can change without notice.

1.3. Background and Literature review

Cloud deployment models offer different approaches to these challenges. As shown in Figure 1.1, there are four primary deployment models: Public clouds are operated by cloud service providers (CSPs) and offer resources to multiple clients with high scalability but lower security and privacy. Private clouds are dedicated to a single organization, providing higher security and complete control but with limited scalability and higher upfront costs. Community clouds serve multiple organizations with shared requirements, offering moderate security and costs. Hybrid clouds combine public and private characteristics, providing flexibility but with increased complexity.[1] Show Image Figure 1.1: Comparison of Cloud Deployment Models

Deployment models	Holder	Security	Scalability	Cost
Private Cloud	Single private organization	Higher than other deployment models	Limited	High
Community Cloud	Two or more private organizations with identical requirements	Lower than private cloud and higher than public and hybrid cloud	Limited	Medium
Public Cloud	Cloud service provider (CSP)	Lower than other deployment models	Very High	Pay-per-use
Hybrid Cloud	CSP and private organizations	Lower than private and community cloud and higher than public cloud	High	Pay-per-use

Figure 1.1: comparing cloud model

For individuals and small organizations prioritizing data privacy and cost-effectiveness, private cloud solutions present an attractive alternative. However, commercial private cloud systems and Network Attached Storage (NAS) devices often cost $300-1000 or more, making them inaccessible to many potential users.[3]

This project addresses these challenges by developing a low-cost private cloud storage system using Raspberry Pi as the server hardware. Raspberry Pi is a credit-card-sized computer costing $35-75 depending on the model, consuming minimal power (approximately 5-15 watts), and capable of running full-featured server software. By connecting affordable external storage devices (such as USB hard drives

or SSDs) to the Raspberry Pi, users can create expandable storage systems at a fraction of the cost of commercial alternatives.[2]

The system is implemented using modern web technologies: Node.js provides the backend server functionality, handling file operations, user authentication, and data management; React creates a responsive, user-friendly web interface accessible from any device with a browser. Security features include user authentication, password hashing, and optional file encryption to protect sensitive data[4]. The system operates within the user's local network, ensuring complete data control, with optional configuration for secure remote access through VPN or port forwarding.

This approach offers several advantages: significantly lower initial investment compared to commercial solutions, no recurring subscription fees, complete data privacy and control, energy-efficient operation suitable for 24/7 use, educational value for learning about cloud computing and server administration, and scalability limited only by available storage devices. While recognizing limitations such as lower performance compared to enterprise solutions, dependency on local network infrastructure, and manual maintenance requirements, this project demonstrates that private cloud storage can be accessible and practical for budget-conscious users who value data privacy.

1.4. The Aims of the Project

The primary objectives of this project are:

1. To create a low-cost private cloud storage solution :

- Utilize Raspberry Pi as an affordable alternative to expensive commercial NAS systems and cloud subscriptions, reducing the total cost of ownership to under $100-150 for hardware
- Demonstrate that effective cloud storage solutions need not require enterprise-grade equipment or ongoing subscription fees

- Provide a replicable model that individuals and small organizations can implement independently

2. To provide secure local data storage with complete user control :

- Ensure all files are stored within the user's local network infrastructure, eliminating third-party access to sensitive data
- Implement authentication mechanisms to prevent unauthorized access
- Use hash-based file integrity verification to detect data corruption or tampering
- Give users full ownership and control over their data, including backup strategies and retention policies

3. To explore energy-efficient and sustainable computing solutions :

- Leverage Raspberry Pi's low power consumption (5-15 watts) compared to traditional servers (100-300 watts) to demonstrate eco-friendly cloud infrastructure
- Create a system suitable for continuous 24/7 operation without significant electricity costs
- Promote awareness of energy-efficient alternatives in personal computing

Goals:

To achieve these objectives, the project will accomplish the following specific goals:

1. Build a fully functional private cloud storage server

- Set up Raspberry Pi with appropriate operating system and server software
- Configure external storage device for file management
- Develop backend API using Node.js to handle file operations (upload, download, delete, rename, move)
- Implement user authentication and authorization system
- Create database structure for managing user accounts, file metadata, and access permissions

- Develop folder hierarchy and organization system
- Implement search functionality to locate files by name, type, or date

2. Design and implement an intuitive user interface

- Create responsive web application using React framework
- Design interface for file browsing, uploading, and downloading
- Implement drag-and-drop functionality for easy file uploads
- Provide progress indicators for file transfers
- Create user dashboard showing storage usage and recent activities
- Ensure mobile compatibility for access from smartphones and tablets

3. Provide both local and remote access capabilities

- Configure system for seamless access within local network
- Document methods for secure remote access (VPN setup or port forwarding with HTTPS encryption)
- Implement dynamic DNS configuration for consistent external access
- Ensure security measures are in place for remote connections

4. Evaluate system performance and document limitations

- Conduct systematic performance testing measuring upload and download speeds for various file sizes (50KB, 500KB, 1MB, 5MB, 25MB)
- Test with different file compression levels to analyze efficiency trade-offs
- Evaluate system behavior under different load conditions (single user vs. multiple concurrent users)
- Measure network throughput limitations imposed by Raspberry Pi hardware
- Document maximum storage capacity, concurrent user limits, and other practical constraints

- Compare performance against commercial cloud services to provide realistic expectations
- Identify optimization opportunities and potential improvements for future development

Chapter Two
Design and Requirements

Chapter 2

Design and Requirements

2.1 Introduction

Archive functions as an independent, self-contained repository for the archiving and distribution of timeseries data. It is a technical system that must meet specific non-functional requirements and consists of both hardware and software components. In order to adhere to the principles of the Raspberry Pi movement and optimize its potential for future adoption, Archive must be developed using open-source tools and frameworks and be extendable. Hardware support for Archive should be inexpensive and resistant to network and power outages. This will make it possible to use it in isolated areas or in impoverished nations. Installing, using, and maintaining the entire Archive system should only require minimal technical expertise. [6]

2.2 System block diagram

The suggested system uses a Raspberry Pi 3 and a client-server architecture to construct a private cloud storage solution. The system enables cloud services to be accessed by desktop and mobile clients over a VPN (TailScale), which controls network communication and routes traffic to the Raspberry Pi server, is used to send client requests. The Raspberry Pi, which runs an operating system based on Linux, serves as the main server for cloud storage.
To process HTTP/HTTPS requests effectively and securely, the server makes use of Nginx as the web server. User authentication, file synchronization, file sharing, and metadata management are among the essential cloud features offered by a customized Nextcloud installation. The system guarantees well-organized file management and safe access control. The two parts of storage are an external USB HDD/SSD that stores user data and an SD card that houses the operating system and server software. System scalability and stability are improved by this division.
All things considered, the architecture offers a private cloud storage solution that is

affordable, energy-efficient, and privacy-focused, making it appropriate for small-scale deployments.[7] Figure 2.1 shows System block diargram

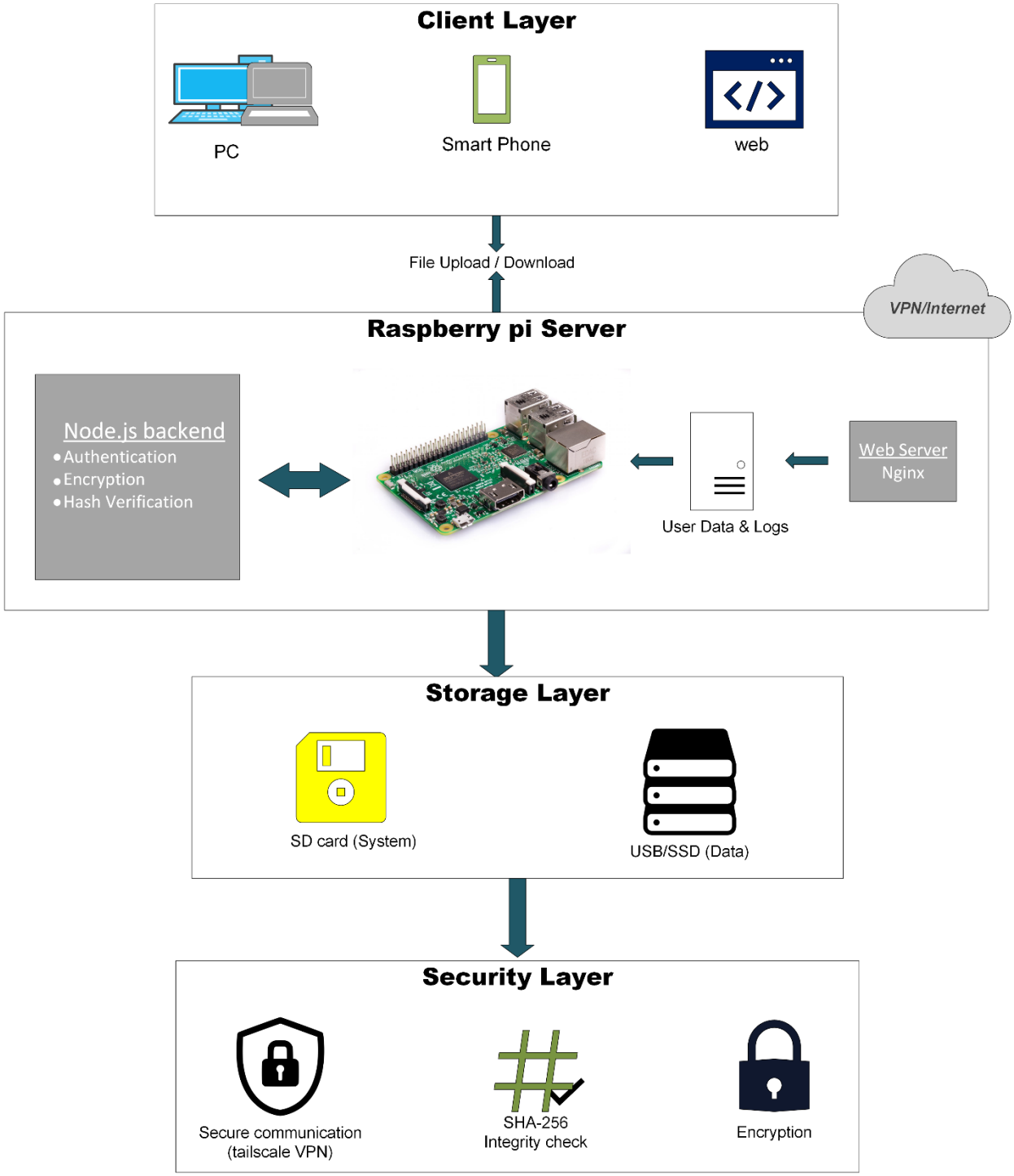

Figure 2.1: block diagram of proposed system

2.3 Hardware Requirements

The proposed system hardware consists of:

- Raspberry Pi 3 B: A device which is used for low computing operations. It serves the purpose of a minicomputer. It can be used as a server as shown in Figure 2.1.

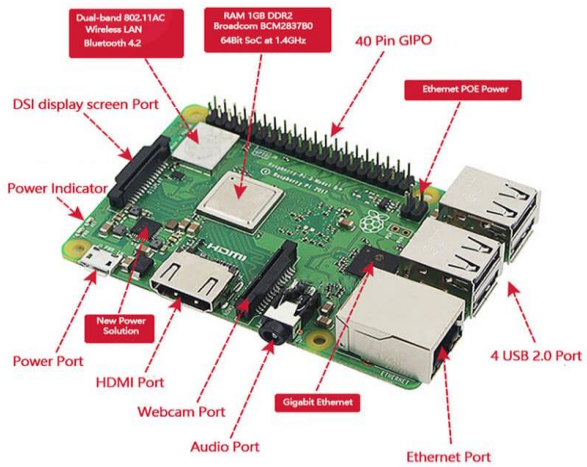

Figure 2.1 architecture of Raspberry pi 3

MicroSD Card: MicroSD card serves the purpose of rom of the raspberry pi. Linux OS is installed on this microSD card.

Hard Drive / USB: To establish a centralized cloud storage system, an external hard drive is required. To gain access to the stored files, the disk needs to be linked and mounted before use.

Client Devices: Devices to access the cloud such as ….

2.4 Software Requirements

- **Raspberry Pi OS (Linux-based):** Operating system installed on the Raspberry Pi

- **Web Server (Nginx)**: NGINX is a web server with an event-driven architecture that is widely recognized for its efficiency in handling large-scale requests and is commonly used as a load balancer in modern web systems [4]

- **Data Base (SQL lit 3):** Manage users and files meta data, SQL lit is a lightweight relational database management system (RDBMS) that stores the entire database in a single file.

- **Cloud Application Logic (Node.js):** Handles file upload, download, sharing, and authentication.

- **Tailscale:** is a modern VPN (Virtual Private Network) that lets your devices securely connect to each other as if they are on the same local network, even when they are in different locations. It is built on WireGuard, which is a very fast and secure VPN protocol.[5]

2.5 System Architecture

The proposed system architecture describes the structural design of the personal cloud storage system implemented using a Raspberry Pi 3. The architecture follows a client–server model and supports hybrid access, allowing users to access the system through the local network and, when required, remotely through the Internet. The system is designed to be low-cost, secure, and easy to deploy while providing core cloud storage functionalities such as file upload, download, and management. Figure 2.2 shows proposed system flowchart[8]

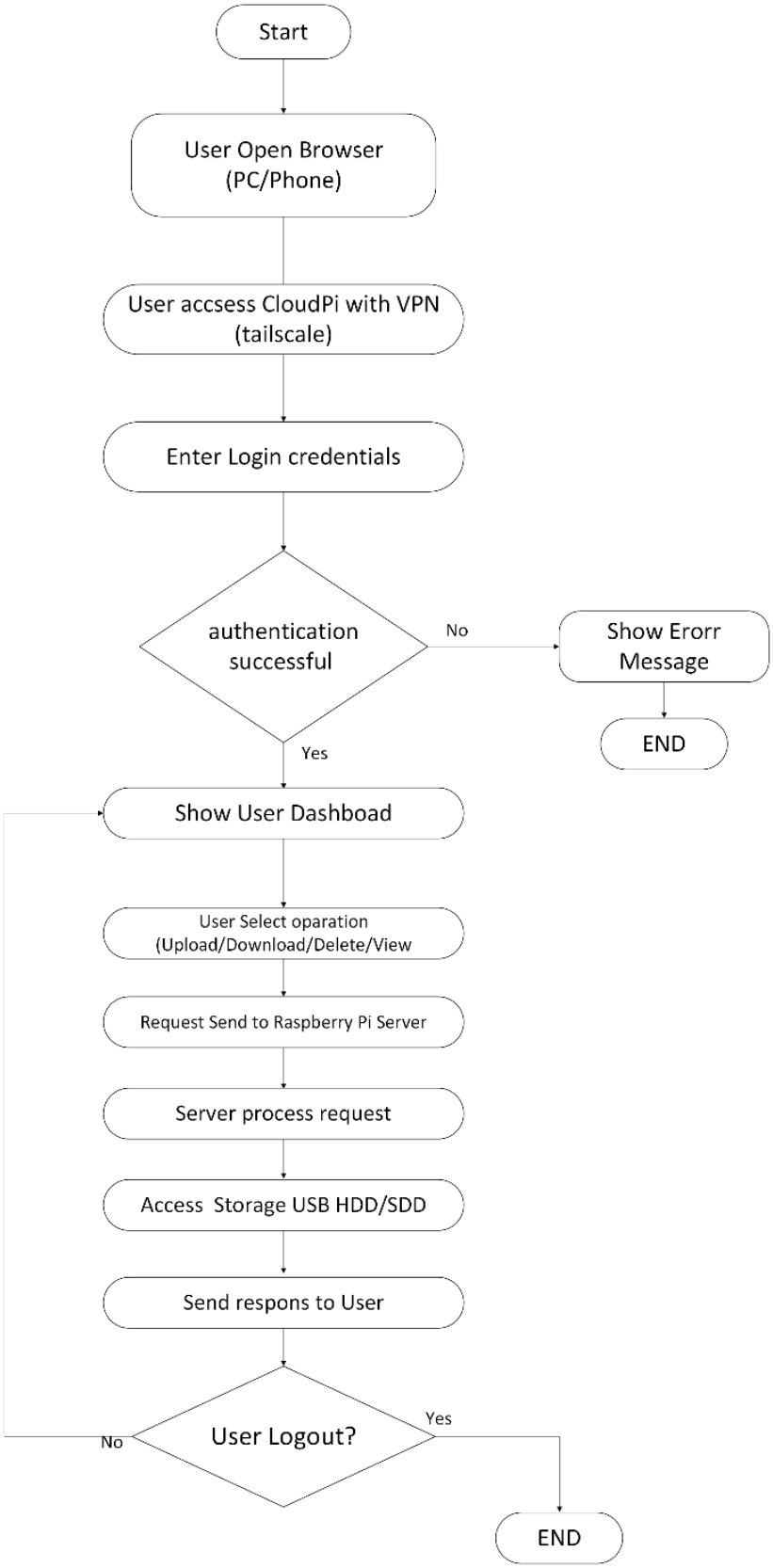

Figure 2.2 proposed system flowchart

Chapter Three
Implementation

Chapter 3

Implementation

3.1 Introduction

This chapter outlines the step-by-step procedure executed to configure a Raspberry Pi 3 as a headless server and web server. The goal of this chapter is to present how prepare the environment for hosting a custom-built cloud storage service. The setup steps include OS installation, remote access configuration, web server deployment, external storage mounting, and memory optimization via ZRAM. As explained in below steps:

3.2Operating System Flashing

Tool Used: Raspberry Pi Imager OS Selected: Raspberry Pi OS Lite (64-bit) The operating system was flashed onto the Micro SD card using the Raspberry Pi Imager. Before writing the image, the "Advanced Options" menu was accessed to pre-configure the following settings for a headless boot as shown in figure 3.1,3.2,3.3:

Hostname: Set to a unique identifier for the network

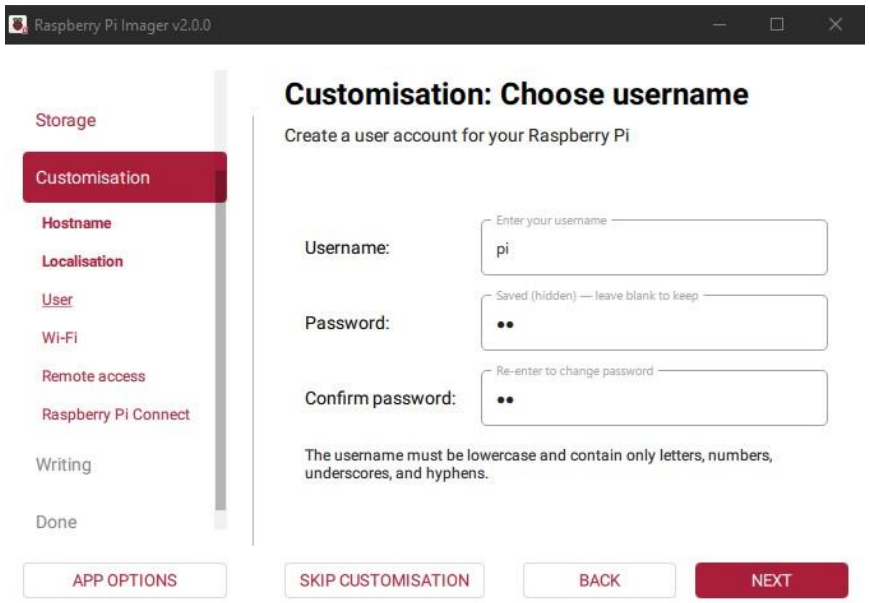

Figure 3.1: Step -1-

Network: Configured Wi-Fi credentials (SSID and Password) to ensure immediate network connectivity upon boot.

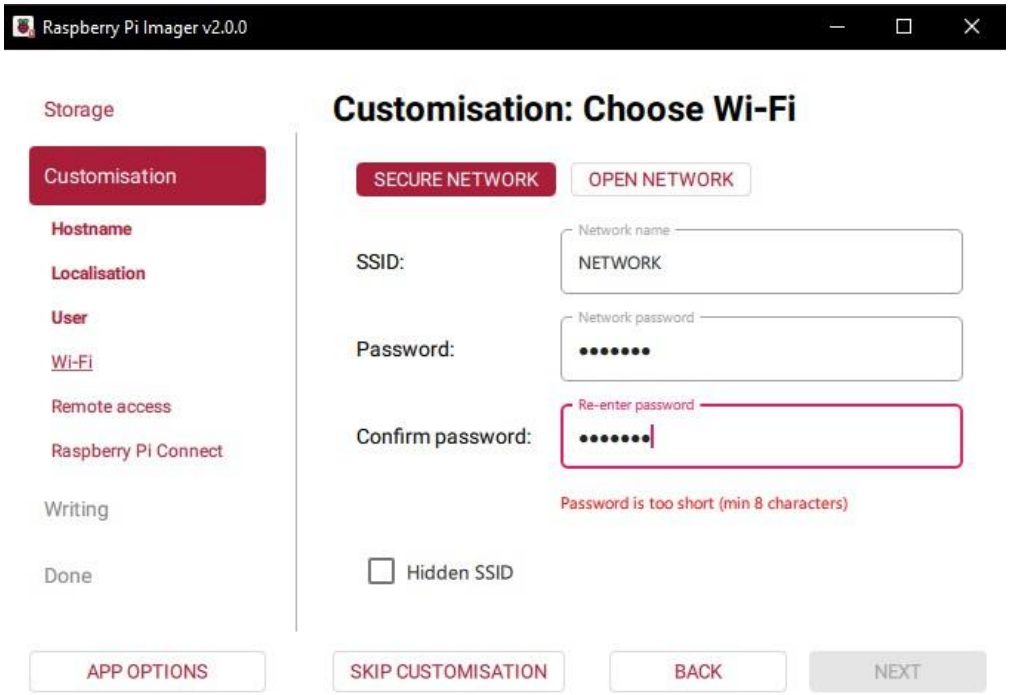

Figure 3.2: Step -2-

SSH: Enabled SSH explicitly with password authentication.

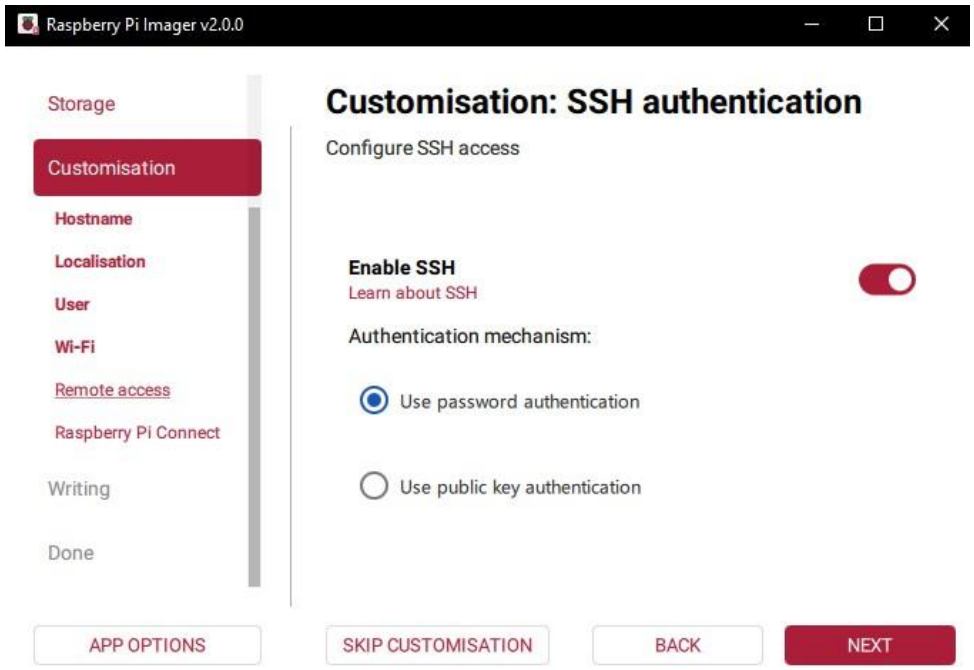

Figure 3.3 : Step -3-

3.2.1 Initial Boot and Remote Connection

After flashing, the SD card was inserted into the Raspberry Pi 3, and the device was powered on.

- Connection Method: SSH (Secure Shell) via laptop terminal.
- Command Executed: SSH (Ip address of the Raspberry Pi)
- Result: Successfully established a secure remote connection to the Raspberry Pi command line interface.

3.2.2 System Maintenance

To ensure system security and software compatibility, the package repositories and installed packages were updated to their latest versions immediately after login.

Commands Executed:

- **sudo apt update**
- **sudo apt upgrade -y**

3.2.4 Web Server Deployment (Nginx)

Nginx was selected and installed to serve the custom cloud application.

Installation: Installed via the package manager using **sudo apt install nginx -y**.

Service Status: Verified that the Nginx service was active and enabled to start on boot using **systemctl status nginx**.

3.2.5 External Storage Configuration

A USB flash drive was mounted to serve as the primary data store for the cloud application, separating user data from the boot OS.

- Identification: Used **lsblk** and **sudo blkid** to identify the partition name and UUID of the USB drive.

- Mount Point: Created a dedicated directory for the mount **(e.g., /mnt/cloud_drive).**
- Persistence: Edited the filesystem table **(/etc/fstab**) to ensure the drive mounts automatically upon system reboot.
- Configuration added: **/dev/sda1 /mnt/usb exfat defaults,nofail 0 0**
- Verification: Mounted the drive using sudo mount -a and verified accessibility.

Status: The Raspberry Pi is now fully configured, optimized, and ready for the deployment of the custom cloud application code.

3.3 Application Layer Deployment

This section outlines the step-by-step implementation of the application layer, including the login system, user interface, backend services, and database integration. The goal is to transform the Raspberry Pi from a configured server into a functional private cloud system.

3.3.1 Frontend

- Frontend Setup (React)
- Technology Used: React (JavaScript Library)

The frontend of the application was developed using React to provide a dynamic and responsive user interface.

Steps Executed:

1. Initialized the React project using a standard project generator.
2. Structured the project into reusable components (Login, Dashboard, Files, Profile, etc.).
3. Configured routing to allow navigation between different pages.
4. Connected the frontend to the backend API using HTTP requests (fetch/axios).

3.3.2 Backend

- Backend Setup (Node.js)
- Technology Used: Node.js (with Express)

The backend server was developed to handle authentication, user management, and file operations.

Steps Executed:

1. Initialized a Node.js project.
2. Installed required dependencies (Express, SQLite3, body-parser, etc.).
3. Created a REST API structure.
4. Implemented authentication endpoint:
    - Endpoint: /login
    - Method: POST
    - Function: Validates user credentials against database records.
5. Implemented additional endpoints:
    - User creation (adding new users)
    - Password reset (backup recovery mechanism)
    - File operations (upload, retrieve, delete, share)

3.4.3 Database Configuration (SQLite3)

- Technology Used: SQLite3

SQLite3 was used as a lightweight local database suitable for Raspberry Pi.

Steps Executed:

1. Created a database file on the Raspberry Pi.
2. Defined tables for system functionality:
    - Users table (username, password, ID)
    - Files table (file metadata)
    - Shared files table
    - User settings table
    - Connected the Node.js backend to the SQLite database.

3. Implemented queries for:
    - User authentication
    - File storage and retrieval
    - User management

3.3.4 Dashboard and User Interface Components

After successful login, the user is redirected to the main dashboard interface.

Steps Executed:

1. Developed dashboard layout using React components.
2. Implemented navigation system for different sections.

Main Sections:

1. Login Page: Provides secure system access by verifying user credentials and preventing unauthorized entry.
2. Dashboard: Overview of system and quick access.
3. Files: Displays user-uploaded files.
4. Shared Files: Files shared by other users.
5. Favorites: Frequently accessed files.
6. Trash: Deleted files storage.
7. Profile: User information management.
8. Settings: System preferences configuration.
9. Admin: Provides administrators with tools to manage users, monitor system activity, and control overall system operations.

1.Login Page

The Login Page serves as the entry point to the system, ensuring secure access through user authentication. It requires users to provide valid credentials, such as a username/Password to verify their identity. As shown in Figure 3.4.

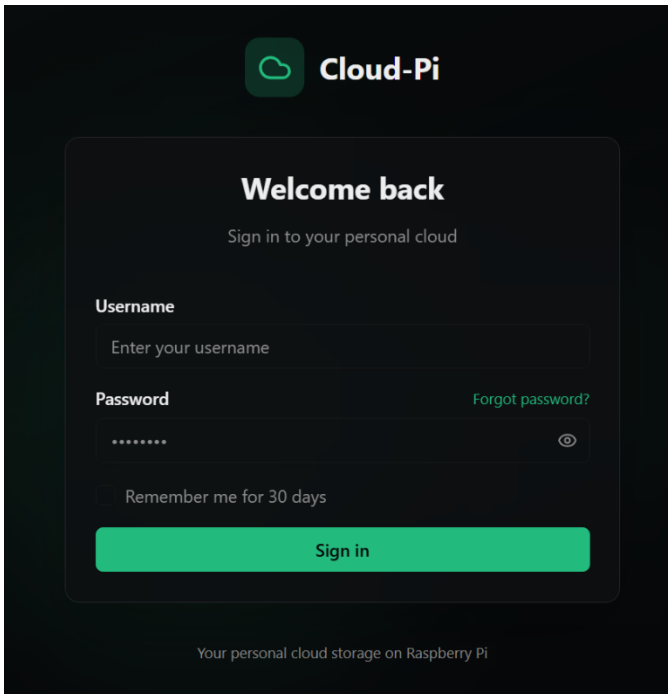

Figure 3.4: Login Page Screen

2.Dashboard

The Dashboard serves as the central hub of the system, providing users with a high-level overview of their activity and resources. It is designed to enhance efficiency by offering quick access to recently used files, key metrics, and important notifications. As Shown in Figure 3.5.

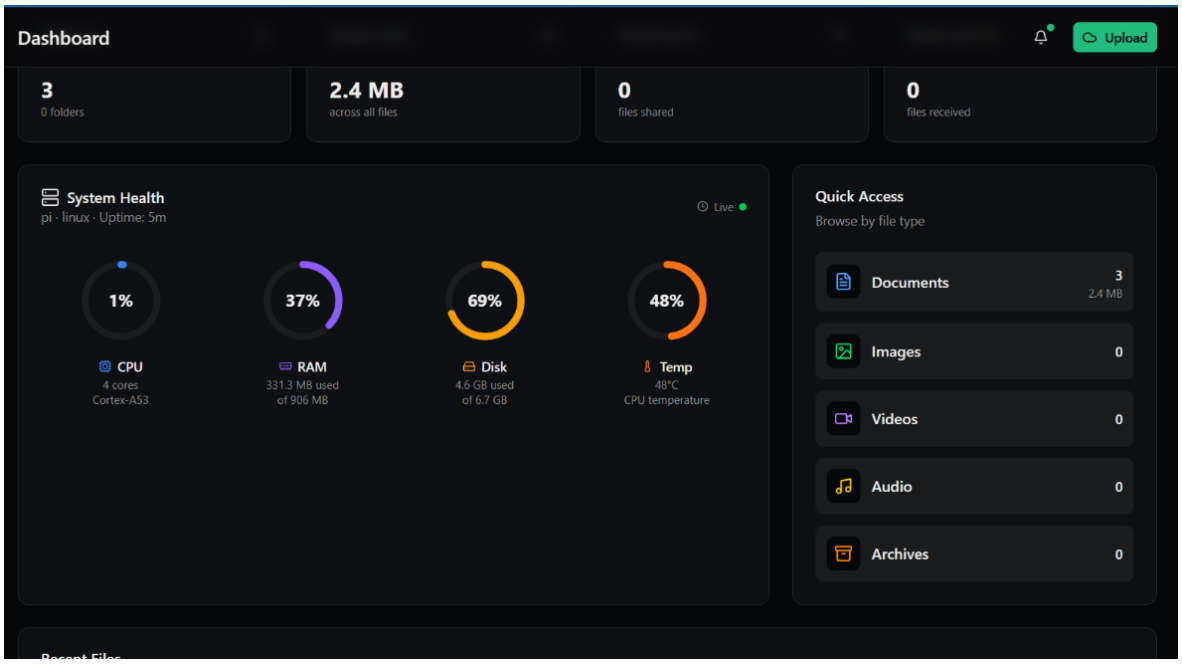

Figure 3.5: Dashboard Page Screen

3.Files

The Files screen presents all content uploaded by the user in an organized manner. It supports file management operations such as viewing, sorting, searching, and categorizing, uploading, downloading, and renaming. As Shown in Figure 3.6

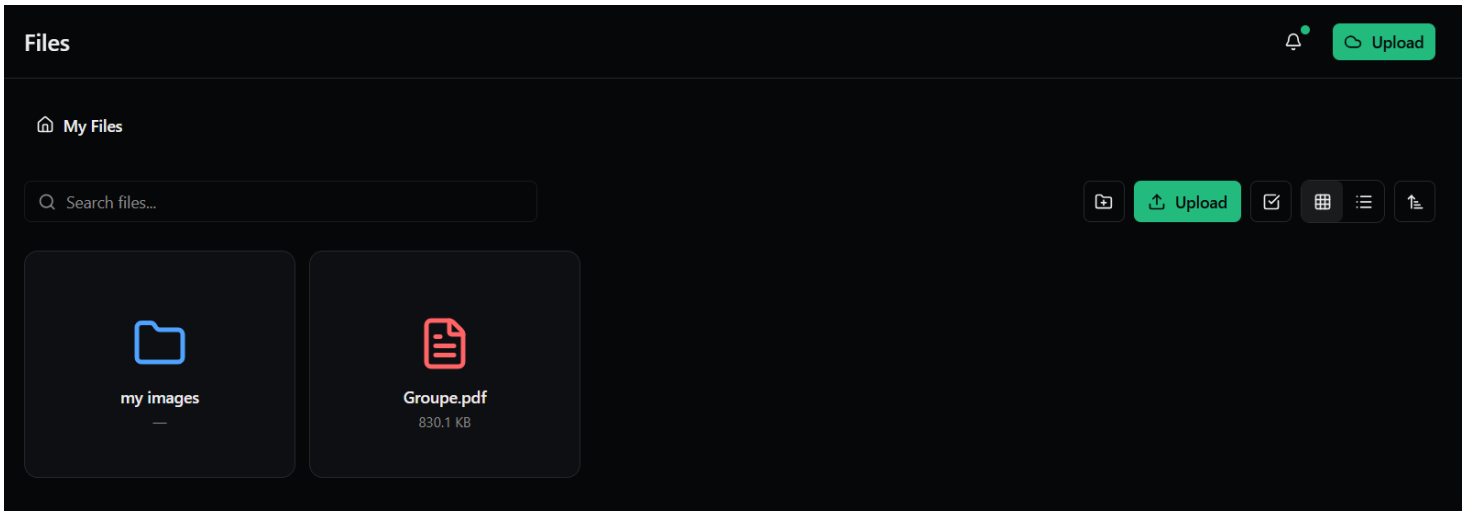

Figure 3.6: Files Page Screen

4.Shared Files

The Shared Files screen displays documents and resources that have been shared with the user by others. It facilitates collaboration by allowing users to access, review, and potentially edit shared content. As shown on Figure 3.7,3.8

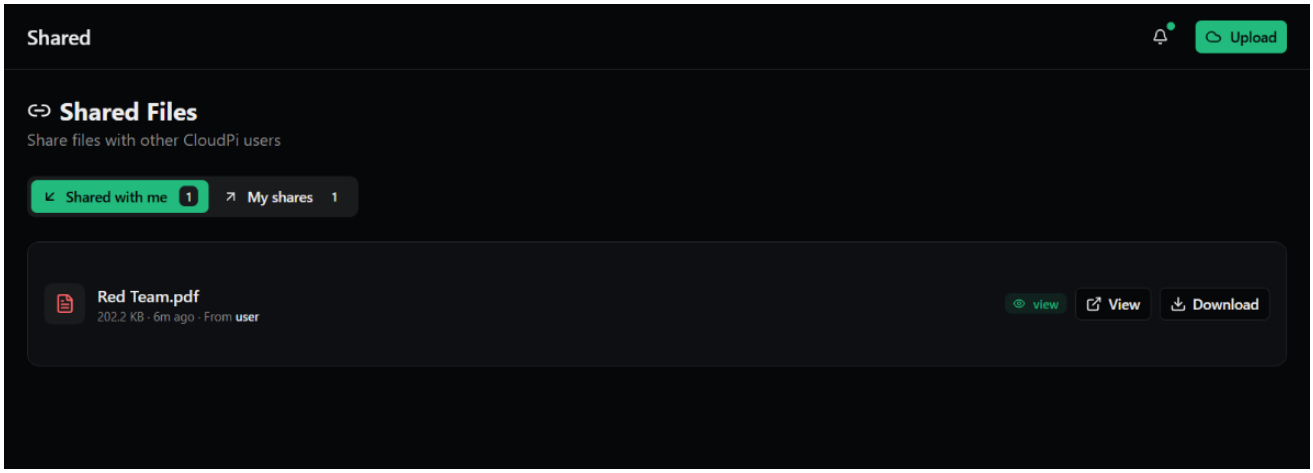

Figure 3.7: Shared Page 1 Screen

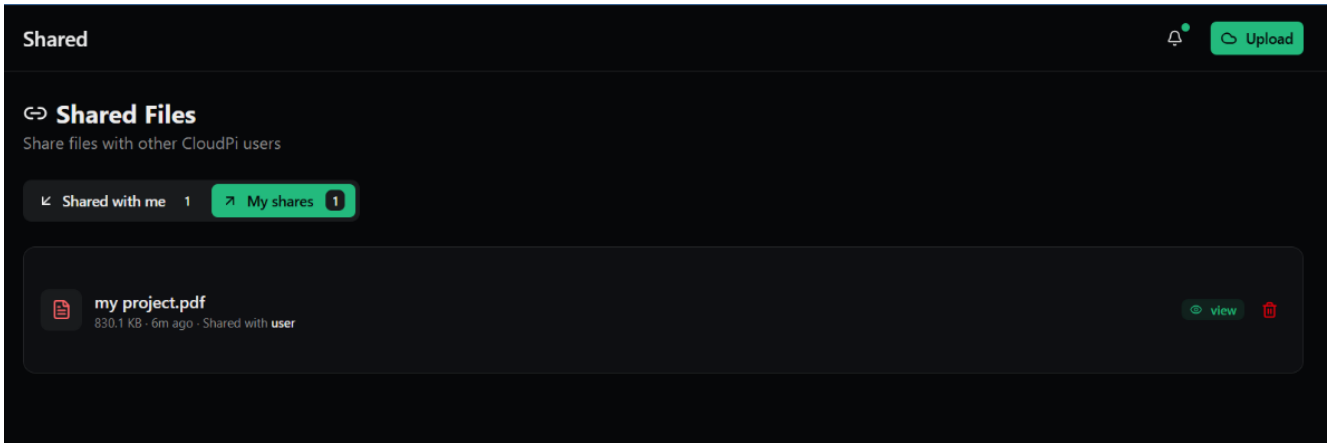

Figure 3.8: Shared Page 2 Screen

5.Starred

The Starred screen allows users to bookmark and quickly retrieve frequently used files. By reducing the time required to search for commonly accessed content, this feature improves user productivity and enhances the overall experience through personalization.as shown in Figure 3.9.

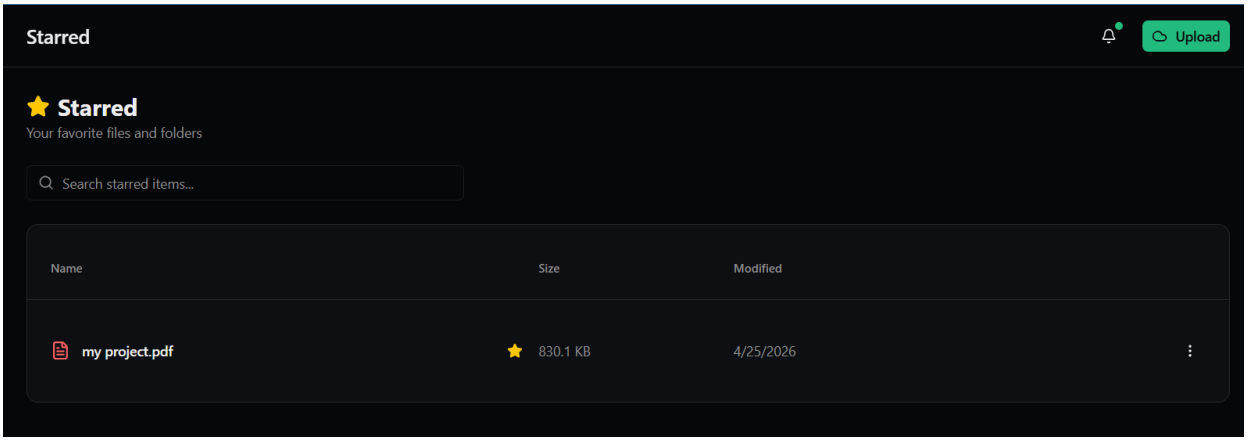

Figure 3.9: Starred Page Screen

6.Trash

The Trash screen acts as a temporary storage area for deleted files. It provides users with the option to restore mistakenly deleted items or permanently remove them.as shown in Figure 3.10.

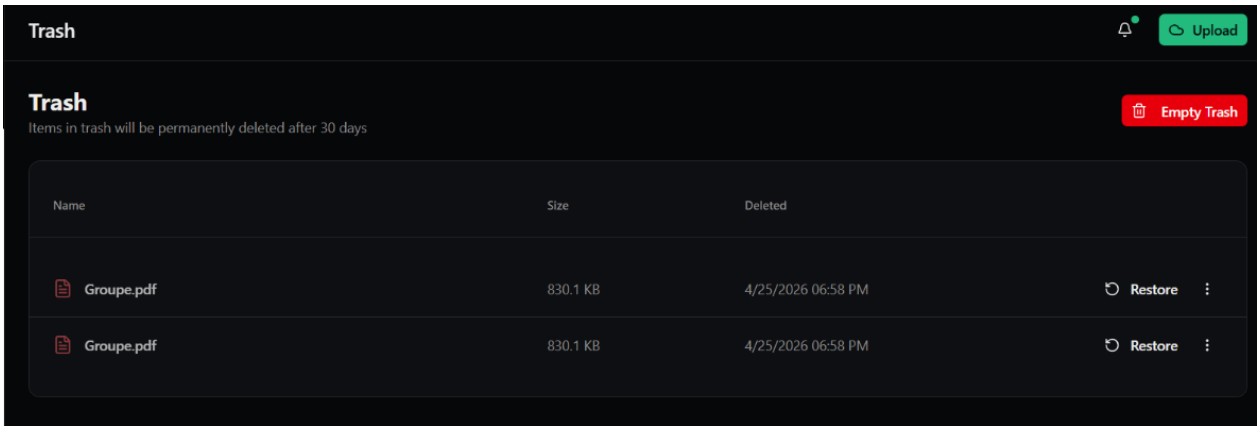

Figure 3.10: Starred Page Screen

7.Profile

The Profile screen enables users to manage their personal information, such as name, Password .As shown in Figure 3.11.

8.Setting

The Settings screen allows users to configure system preferences, including privacy options, and interface customization. It provides flexibility and adaptability .As shown in figure 3.12.

9.Admin

Platform administration restricted to admins — user management (create, delete, reset passwords, assign default storage, set quotas) and a Storage Manager for scanning, mounting, registering, and ejecting USB drives. As shown Figure 3.13

3.4 Performance Evaluation and Resource Usage Analysis

The experimental results show in Table 3.1, 3.2 ,3.3 that the suggested cloud storage system based on Raspberry Pi works well and consistently, no matter how big or small the files are or how much they are compressed. The system has low latency and short upload and download times for small files (50 KB), but the throughput is still limited because it takes longer to handle small data packets.When the file size goes up, the throughput gets a lot better. For example, when transferring medium-sized files (1 MB) and standard-sized files (5 MB), the system can send data at speeds of over 40 MB/s. This means that the system works better when it has to deal with larger amounts of data, because the overhead for communication is lower than the size of the payload.The analysis of CPU utilization indicates that resource consumption escalates with file size and processing complexity, especially when compression is utilized. But CPU usage stays within acceptable limits, usually below 50%, which shows that the Raspberry Pi can handle the workload without slowing down. Memory usage also goes up slowly as files get bigger, but it stays stable overall, which shows that memory management is working well.Also, the concurrent user test shows that the system still has a high success rate (100%) even when there are up to 20 users at the same time. The system still works reliably, even though

response times go up when there is stress, and CPU and RAM usage is moderate.In general, the results show that the proposed system strikes a good balance between performance and resource use, making it a good low-cost cloud storage option for small deployments.

File Size	Compression Level	Upload Time (s)	Download Time (s)	Throughput (MB/s)	CPU Usage (%)	Memory Usage (MB)	Hash Verification
50 KB	None	0.058	0.022	1.23	24%	383	Pass
50 KB	50%	0.068	0.018	1.13	31%	384	Pass
50 KB	75%	0.049	0.015	1.51	22%	386	Pass
500 KB	None	0.055	0.026	12.04	28%	387	Pass
500 KB	50%	0.055	0.026	11.95	27%	384	Pass
500 KB	75%	0.056	0.024	12.25	26%	387	Pass
1 MB	None	0.071	0.057	15.68	51%	395	Pass
1 MB	50%	0.073	0.035	18.39	34%	405	Pass
1 MB	75%	0.079	0.044	16.26	29%	411	Pass
5 MB	None	0.125	0.098	44.78	43%	431	Pass
5 MB	50%	0.154	0.109	38.01	45%	412	Pass
5 MB	75%	0.127	0.105	43.09	45%	435	Pass
25 MB	None	1.006	0.634	30.48	40%	529	Pass
25 MB	50%	0.852	0.551	35.62	38%	520	Pass
25 MB	75%	0.928	0.533	34.22	38%	551	Pass

Table 3.1: Performance Evaluation Table

File Size Category	Test Case	Avg. Upload Speed (MB/s)	Avg. Download Speed (MB/s)	CPU Usage (%)	RAM Usage (MB)	Hash Verification Time (ms)
Small (50KB)	Random data	0.9	2.7	26%	386MB	<5ms
Medium (1MB)	Mixed data	13.5	22.9	38%	411MB	15ms
Standard (5MB)	Mixed data	37.3	48.2	44%	435MB	45ms
Large (25MB)	Repetitive data	27.0	43.9	39%	551MB	180ms

Table 3.2: Summary by File Size Category

Users	Operations	Total Time (s)	Avg Upload (s)	Avg Download (s)	Success Rate	CPU (%)	RAM (MB)
1	Upload/download	0.19	0.066	0.031	1/1	22%	556
5	Mixed operations	0.89	0.259	0.233	5/5	29%	547
10	Mixed operations	2.16	0.842	0.623	10/10	22%	544
20	Stress condition	3.40	0.972	1.233	20/20	27%	484

Table 3.3: Concurrent User Test

3.5 Security Layer Implementation

Security is a fundamental concern in any cloud storage system, particularly in self-hosted environments where the server is physically accessible and data traverses networks of varying trust levels. This section addresses these concerns through a multi-layered security architecture that encompasses data integrity verification, encryption at rest, secure communication, and network-level access control. This section describes the design and implementation of each security component within the CloudPi system.

3.5.1 File Integrity Verification

File integrity verification ensures that data has not been altered, corrupted, or tampered with during storage or transmission[9]. The Secure Hash Algorithm 256-bit (SHA-256), a member of the SHA-2 family published by the National Institute of Standards and Technology (NIST), produces a fixed-size 256-bit (32-byte) hash value for any input. The algorithm is computationally infeasible to reverse, and any modification to the input — even a single bit — results in a fundamentally different hash output, a property known as the avalanche effect (NIST, 2015). In this proposed System, a SHA-256 hash is computed for every file at the point of upload, prior to any encryption. The hash is calculated using Node.js's built-in crypto module, which provides a native binding to OpenSSL's implementation of SHA- 256. The file is read as a stream to maintain memory efficiency when processing large files, and the resulting hexadecimal digest is stored alongside the file metadata in the SQLite database. Upon download, the system recomputes the SHA-256 hash of the file (after decryption, if applicable) and compares it against the stored value. A mismatch indicates that the file has been modified or corrupted since upload, and the system flags this discrepancy to the user through an HTTP response header (X-Integrity-Warning). This approach provides end-to-end integrity assurance without blocking legitimate file access. SHA-256 was selected over alternatives such as MD5 or SHA-1 due to known collision vulnerabilities in those algorithms[10]. SHA-256 remains cryptographically secure with no known practical collision attacks, making it suitable for long-term data integrity verification in production systems.

3.4.3 AES-256-GCM Encryption at Rest

Encryption at rest protects stored data from unauthorized access in scenarios where physical storage media may be compromised — a particularly relevant concern for Raspberry Pi deployments where SD cards and USB drives are easily removable. The Advanced Encryption Standard (AES) with a 256-bit key length, operating in Galois/Counter Mode (GCM), provides both confidentiality and authenticity through authenticated encryption with associated data (AEAD)[11] the proposed system employs AES-256-GCM to encrypt files at rest on the storage medium.
The encryption process proceeds as follows:

1. Key Management: A 256-bit (32-byte) master encryption key is generated using a cryptographically secure random number generator (crypto.randomBytes) upon initial system setup. This key is stored in an environment variable file (.env) on the server and is loaded into memory at runtime. The key is excluded from version control through. gitignore rules.

2. Encryption Process: After a file is uploaded and its SHA-256 hash is computed from the plaintext content, the system encrypts the file in place using AES-256-GCM. A unique 12-byte initialization vector (IV) is generated randomly for each file to ensure that identical files produce different ciphertexts. The encrypted file is stored on disk in the following binary format: [12 bytes: IV] [16 bytes: Authentication Tag] [N bytes: Ciphertext]

3. Decryption Process: Upon download or preview, the system reads the IV and authentication tag from the file header, decrypts the ciphertext using the master key, and verifies the authentication tag. If the tag verification fails — indicating tampering with the encrypted data — the decryption operation is rejected, and an error is returned to the client.

4. Administrative Control: Encryption is configurable through the admin settings panel, allowing the administrator to enable or disable encryption for new uploads without affecting previously stored files. Each file record in the database includes an encrypted flag indicating its encryption status, enabling the system to handle mixed encrypted and unencrypted files transparently.

AES-256-GCM was chosen for the following reasons:

• Security Strength: AES-256 provides a 256-bit key space, which is considered secure against brute-force attacks even by theoretical quantum computing adversaries under Grover's algorithm, which would reduce the effective key strength to 128 bits — still computationally infeasible[12]

• Authenticated Encryption: GCM mode combines encryption and message authentication into a single operation, eliminating the need for a separate HMAC computation and preventing attacks such as padding oracle attacks that affect other modes like CBC

• Performance: GCM mode is parallelizable and benefits from AES-NI hardware acceleration available on many processors, making it suitable for the resource-constrained Raspberry Pi environment.

• Native Support: Node.js's crypto module provides native GCM support through OpenSSL, requiring no additional dependencies.

3.4.4 Nginx Reverse Proxy

A reverse proxy sits between client devices and the backend application server, forwarding client requests to the server and returning the server's responses to the client. This architectural pattern provides several security and performance benefits, including SSL/TLS termination, request filtering, and header management Nginx was selected as the reverse proxy due to its low memory footprint, event driven architecture suited to the Raspberry Pi's limited resources, and its established track record in production environments. Direct exposure of the Node.js application server to the internet is avoided, as web servers like Nginx are specifically hardened against common HTTP-based attacks [13] .

3.4.5 Tailscale VPN — Secure Communication

Virtual Private Networks (VPNs) establish encrypted tunnels between devices, enabling secure communication over untrusted networks. Tailscale is a mesh VPN built on the WireGuard protocol, which provides modern cryptographic primitives including Curve25519 for key exchange, ChaCha20-Poly1305 for symmetric encryption, and BLAKE2s for hashing (Donenfeld, 2017). Unlike traditional VPNs that route all traffic through a central server, Tailscale establishes direct peer-to-peer connections between devices where possible, minimizing latency.

3.6 Conclusions

3.7 Future Directions

پوختە

نوسینی پوختەیەک بە زمانی کوردی لەسەر پڕۆژەکە کە هەمان ناوەڕۆکی (Abstract) هەیە، بەڵام بە زمانی کوردی پاراو، ڕەچاوی خاڵبەندی و ڕێنوسی زمانی کوردی بکرێت.

References

[1] G. K. John Smith, "Study on the interaction mechanism between laser and," *Optics and Laser Technology,* vol. 4, no. 1, pp. 15-25, 2019.